

C++ Object-Oriented Programming

Practical Questions

Instructions:

- Write and execute each program in C++.
- Use meaningful class, object, data member, and function names.
- Add comments wherever necessary.
- The questions progress from basic C++ syntax to introductory OOP concepts.

Practical Questions

A. C++ Basics, Operators and Dynamic Memory

1. Write a C++ program to display “Hello, World!” using cout.
2. Write a C++ program to input two integers using cin and display their sum.
3. Write a C++ program to demonstrate the use of arithmetic operators (+, -, *, /, %).
4. Write a C++ program to demonstrate relational operators using two numbers.
5. Write a C++ program to demonstrate logical operators (&&, ||, !).
6. Write a C++ program to swap two numbers using a temporary variable.
7. Write a C++ program to find the largest of two numbers using if-else.
8. Write a C++ program to check whether a number is even or odd.
9. Write a C++ program to calculate the area of a rectangle using user input.
10. Write a C++ program to input marks of three subjects and calculate total and average.

B. Classes, Objects, Attributes and Methods

11. Write a C++ program to define a simple class Student with one data member name and display it using a member function.
12. Create a class Rectangle with length and breadth as data members and a method to calculate area.
13. Create a class Calculator with methods for addition, subtraction, multiplication, and division.
14. Create a class Car with attributes brand and model and a method to display the car details.
15. Create a class Employee with name and salary attributes and a method to display employee information.
16. Create a class Circle with radius as an attribute and a method to calculate its area.
17. Create a class Book with title and price attributes and display the book details.
18. Create a class Mobile with brand and price attributes and initialize them through a member function.
19. Write a program to create two objects of a Student class and display their different data.

C. Encapsulation, Information Hiding and ADT

20. Create a class BankAccount with a private balance and public deposit() and displayBalance() methods.
21. Create a class Person with private age and public setter and getter methods to demonstrate encapsulation.
22. Create a class Temperature with a private Celsius value and methods to set and display the temperature.
23. Create a class Product with private price and a method to update and display the price.
24. Create a simple abstract data type class Counter with private count and public increment() and getCount() methods.
25. Create a class Student with private marks and a method that returns the marks only if they are within a valid range.
26. Create a class Account with private accountNumber and balance and public methods to deposit and withdraw money.